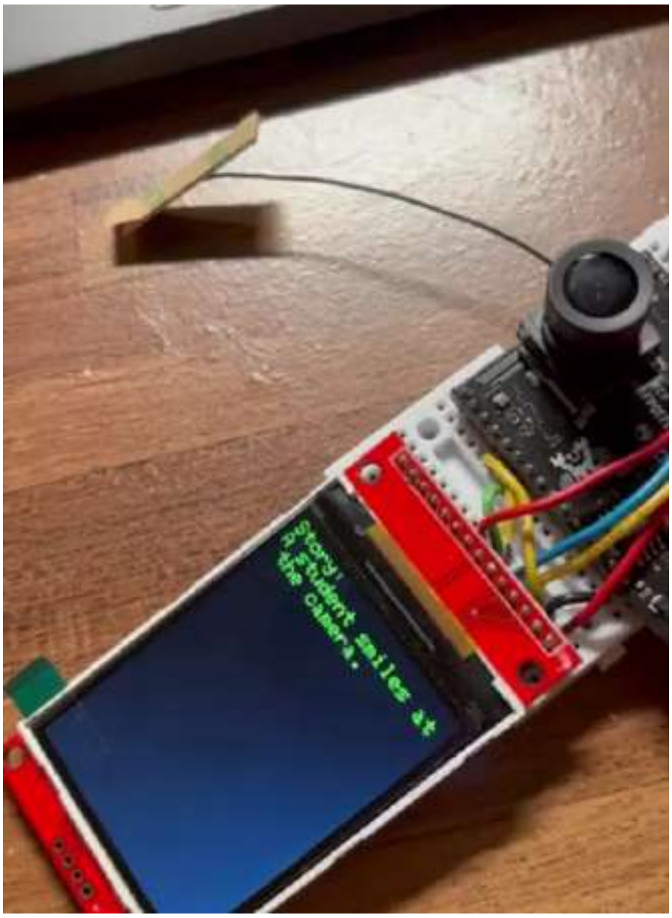
本系統可透過按鈕拍攝照片，並用 AI 分析畫面內容，自動生成簡短故事。系統會先於 LCD 顯示英文故事內容，再利用 TTS 語音播放中文故事，實現簡易的 AI 看圖說故事互動功能。



https://youtube.com/shorts/89vE1KZBm-Y?feature=share

```
String Gemini_key = "先碼掉";

char wifi_ssid[] = "321321321";
char wifi_pass[] = "12345678901";

#include <WiFi.h>
#include <WiFiUdp.h>
#include "GenAI.h"
#include "VideoStream.h"
#include "SPI.h"
#include "AmebaILI9341.h"
#include "AmebaFatFS.h"
```

```
WiFiSSLClient client;
GenAI llm;
GenAI tts;

AmebaFatFS fs;
String mp3Filename = "story_tts.mp3";

VideoSetting config(320, 320, CAM_FPS, VIDEO_JPEG, 1);
#define CHANNEL 0

uint32_t img_addr = 0;
uint32_t img_len = 0;

const int buttonPin = 1;
const bool USE_INTERNAL_PULLUP = false;
const int PRESSED_STATE = USE_INTERNAL_PULLUP ? LOW : HIGH;

String prompt_msg =
"請根據這張圖片創作一段約 50 字的繁體中文短故事，"
"再提供一段簡短英文翻譯。"
"請嚴格使用以下格式輸出："
"CN:中文故事"
"EN:English translation";

#define TFT_RESET 5
#define TFT_DC    4
#define TFT_CS    SPI_SS

AmebaILI9341 tft = AmebaILI9341(TFT_CS, TFT_DC, TFT_RESET);
#define ILI9341_SPI_FREQUENCY 20000000

void initWiFi()
{
    for (int i = 0; i < 2; i++) {
        WiFi.begin(wifi_ssid, wifi_pass);

        delay(1000);
        Serial.println("");
```

```
        Serial.print("Connecting to WiFi: ");
        Serial.println(wifi_ssid);

        uint32_t StartTime = millis();

        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");

            if ((StartTime + 10000) < millis()) {
                break;
            }
        }

        if (WiFi.status() == WL_CONNECTED) {
            Serial.println("");
            Serial.println("WiFi connected.");
            Serial.print("IP address: ");
            Serial.println(WiFi.localIP());
            Serial.println("");
            break;
        } else {
            Serial.println("");
            Serial.println("WiFi connection failed.");
        }
    }
}

void init_tft()
{
    tft.begin();
    tft.setRotation(2);
    tft.clr();
    tft.setCursor(0, 0);
    tft.setForeground(ILI9341_GREEN);
    tft.setFontSize(2);
}
```

```
void waitButtonRelease()
{
    while (digitalRead(buttonPin) == PRESSED_STATE) {
        delay(20);
    }

    delay(200);
}

void sdPlayMP3(String filename)
{
    fs.begin();

    String filepath = String(fs.getRootPath()) + filename;
    File file = fs.open(filepath, MP3);

    file.setMp3DigitalVol(175);
    file.playMp3();
    file.close();

    fs.end();
}

String getCNText(String aiText)
{
    int cnStart = aiText.indexOf("CN:");
    int enStart = aiText.indexOf("EN:");

    if (cnStart != -1 && enStart != -1) {
        return aiText.substring(cnStart + 3, enStart);
    }

    return aiText;
}

String getENText(String aiText)
{
    int enStart = aiText.indexOf("EN:");
```

```
    if (enStart != -1) {
        return aiText.substring(enStart + 3);
    }

    return "AI generated a short story.";
}

void setup()
{
    Serial.begin(115200);
    delay(1000);

    Serial.println("System start.");

    SPI.setDefaultFrequency(ILI9341_SPI_FREQUENCY);

    init_tft();

    tft.println("Booting...");

    initWiFi();

    tft.println("WiFi OK");

    config.setRotation(0);

    Camera.configVideoChannel(CHANNEL, config);
    Camera.videoInit();
    Camera.channelBegin(CHANNEL);

    delay(2000);

    Camera.printInfo();

    tft.println("Camera OK");

    pinMode(buttonPin, USE_INTERNAL_PULLUP ? INPUT_PULLUP : INPUT);
```

```
    pinMode(LED_B, OUTPUT);

    tft.clr();
    tft.setCursor(0, 0);
    tft.println("AI Vision");
    tft.println("System Ready");
    tft.println("Press Button");
}

void loop()
{
    if (digitalRead(buttonPin) == PRESSED_STATE) {

        waitButtonRelease();

        Serial.println("");
        Serial.println("Button pressed.");

        tft.clr();
        tft.setCursor(0, 0);
        tft.println("Prepare");
        tft.println("Capture");

        for (int count = 3; count > 0; count--) {
            Serial.print("Countdown: ");
            Serial.println(count);

            tft.clr();
            tft.setCursor(0, 0);
            tft.println("Capture in");
            tft.println(count);

            digitalWrite(LED_B, HIGH);
            delay(500);
            digitalWrite(LED_B, LOW);
            delay(500);
        }
```

```
    Serial.println("Capturing image...");

    tft.clr();
    tft.setCursor(0, 0);
    tft.println("Capturing");

    img_addr = 0;
    img_len = 0;

    Camera.getImage(CHANNEL, &img_addr, &img_len);

    Serial.print("Image address: ");
    Serial.println(img_addr);

    Serial.print("Image length: ");
    Serial.println(img_len);

    if (img_len == 0 || img_addr == 0) {
        Serial.println("Camera capture failed.");

        tft.clr();
        tft.setCursor(0, 0);
        tft.println("Camera");
        tft.println("Failed");

        delay(2000);
        return;
    }

    Serial.println("Sending image to Gemini Vision...");

    tft.clr();
    tft.setCursor(0, 0);
    tft.println("AI");
    tft.println("Analyzing");

    String aiText = llm.geminivision(
        Gemini_key,
```

```
        "gemini-2.0-flash",
        prompt_msg,
        img_addr,
        img_len,
        client
    );

    Serial.println("");
    Serial.println("AI response:");
    Serial.println(aiText);

    if (aiText.length() == 0) {
        Serial.println("AI response is empty.");

        tft.clr();
        tft.setCursor(0, 0);
        tft.println("AI Failed");
        tft.println("Empty Text");

        delay(2000);
        return;
    }

    if (aiText.indexOf("Gemini error") != -1 ||
        aiText.indexOf("Quota exceeded") != -1 ||
        aiText.indexOf("TooManyRequests") != -1 ||
        aiText.indexOf("API key") != -1) {

        Serial.println("Gemini API error detected. Stop TTS.");

        tft.clr();
        tft.setCursor(0, 0);
        tft.println("Gemini Error");
        tft.println("Check Key");
        tft.println("or Quota");

        delay(3000);
        return;
```

```
    }

    String cn_story = getCNText(aiText);
    String en_story = getENText(aiText);

    cn_story.trim();
    en_story.trim();

    Serial.println("");
    Serial.println("Chinese story:");
    Serial.println(cn_story);

    Serial.println("");
    Serial.println("English story:");
    Serial.println(en_story);

    tft.clr();
    tft.setCursor(0, 0);
    tft.println("Story:");
    tft.println(en_story);

    delay(3000);

    Serial.println("Converting Chinese story to speech...");

    tft.clr();
    tft.setCursor(0, 0);
    tft.println("TTS");
    tft.println("Generating");

    tts.googletts(mp3Filename, cn_story, "zh-TW");

    delay(500);

    Serial.println("Playing MP3...");

    tft.clr();
    tft.setCursor(0, 0);
```

```
        tft.println("Playing");
        tft.println("Audio");

        sdPlayMP3(mp3Filename);

        Serial.println("Process finished.");

        tft.clr();
        tft.setCursor(0, 0);
        tft.println("Finished");
        tft.println("Press Again");
    }
}
```